

Linux Hardening – Step-by-Step Lab Guide

Assignment: Endpoint Security & Linux Hardening
Submission Deadline: November 29th, 2025 (Midnight)

Student Name: Mohammad Salem Hassani
Course: Cybersecurity / Network Defense
Instructor: Asen Grozdanov
GitHub Repository: github.com/salem228h/Cybersecurity-Portfolio
GitHub Username: salem228h
Lab Date: November 2025

Lab Overview

Purpose and Objectives

The Linux Hardening Lab is designed to provide students with hands-on experience in securing Linux systems through systematic kernel management, package maintenance, SSH configuration, and firewall implementation. This lab focuses on **Endpoint Security** best practices, enabling students to understand critical hardening techniques used in production environments.

Key Learning Outcomes

By completing this lab, students will:

- Understand kernel versioning and system architecture
- Manage system repositories and dependencies effectively
- Identify and remove obsolete kernel versions securely
- Configure SSH for enhanced security
- Implement firewall rules using both CLI and GUI tools
- Apply defense-in-depth principles to Linux systems

Lab Environment

- **OS:** Linux (Ubuntu)
- **Kernel Version:** 6.14.0 series
- **Tools:** UFW, GFW, SSH, APT package manager
- **Prerequisites:** Basic Linux command-line knowledge, sudo access

Exercise 1 – System Version Verification

Objective

Identify the active kernel version and system architecture of the Linux system.

Command

```
## uname -mrs
```

Code Description and Design

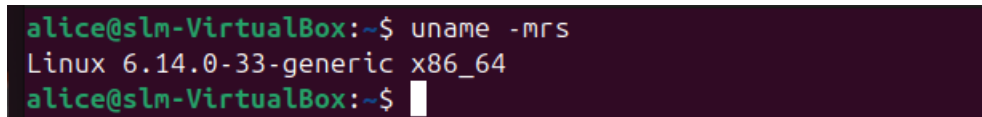
What This Command Does:

The `uname` command is fundamental for system reconnaissance. This command retrieves critical system information:

- **uname** (Unix Name): Core utility that displays system information
 - **-m** (Machine Hardware Name): Returns processor architecture (x86_64, arm64, i386, etc.). Essential for determining software compatibility and security policies specific to CPU type
 - **-r** (Kernel Release): Shows the exact kernel version (e.g., 6.14.0-33-generic). Critical for identifying security patches and known vulnerabilities
 - **-s** (Kernel Name): Displays the OS name (Linux, BSD, etc.). Useful in heterogeneous environments

Security Significance:

Understanding your kernel version is the first step in hardening. You can cross-reference this version against security bulletins to identify pending patches.

A terminal window with a dark purple background. The prompt is 'alice@slm-VirtualBox:~\$'. The command 'uname -mrs' has been entered and executed. The output is 'Linux 6.14.0-33-generic x86_64'. The prompt is now 'alice@slm-VirtualBox:~\$' with a cursor at the end.

```
alice@slm-VirtualBox:~$ uname -mrs
Linux 6.14.0-33-generic x86_64
alice@slm-VirtualBox:~$
```

Capture the terminal after executing the above command

Exercise 2 – Local Repository Maintenance

Objective

Update system packages, apply security patches, and clean unnecessary cache to maintain system integrity and free disk space.

Commands

Step 1: Update Package Lists

```
## sudo apt update
```

Code Description:

- **sudo**: Execute with superuser privileges (required for system operations)
- **apt**: Advanced Package Tool - Debian/Ubuntu package manager
- **update**: Refreshes package lists from configured repositories
 - Downloads metadata about available packages
 - Checks for new versions and security updates
 - Does NOT install packages yet

Why It Matters: Repository cache can become stale. Running `apt update` ensures you're installing from current, secure sources.

Step 2: Upgrade System

```
## sudo apt upgrade -y
```

Code Description:

- **upgrade**: Installs available updates for installed packages
 - Maintains dependency compatibility
 - Never removes packages
 - Safe way to apply security patches
- **-y**: Automatic "yes" to prompts - non-interactive mode for scripting/automation

Important: Unlike `apt full-upgrade` or `apt dist-upgrade`, this command won't automatically remove conflicting packages.

Step 3: Clean Package Cache

```
## sudo apt-get autoclean
```

Code Description:

- **apt-get**: Lower-level package tool (older syntax, still widely used)
- **autoclean**: Removes cached packages that are no longer available in repositories
 - Frees disk space (typically 100-500 MB depending on usage)
 - Keeps recently used packages for faster reinstallation
 - More conservative than `clean` (which removes ALL cached packages)

```

alice@slm-VirtualBox:~$ sudo apt update
[sudo] password for alice:
Get:1 http://security.ubuntu.com/ubuntu noble-security InRelease [126 kB]
Hit:2 http://ca.archive.ubuntu.com/ubuntu noble InRelease
Hit:3 https://packages.wazuh.com/4.x/apt stable InRelease
Get:4 http://ca.archive.ubuntu.com/ubuntu noble-updates InRelease [126 kB]
Get:5 http://ca.archive.ubuntu.com/ubuntu noble-backports InRelease [126 kB]
Get:6 http://security.ubuntu.com/ubuntu noble-security/main amd64 Components [21.5 kB]
Get:7 http://ca.archive.ubuntu.com/ubuntu noble-updates/main amd64 Components [175 kB]
Get:8 http://security.ubuntu.com/ubuntu noble-security/restricted amd64 Components [212 B]
Get:9 http://security.ubuntu.com/ubuntu noble-security/universe amd64 Components [52.2 kB]
Get:10 http://security.ubuntu.com/ubuntu noble-security/multiverse amd64 Components [212 B]
Get:11 http://ca.archive.ubuntu.com/ubuntu noble-updates/restricted amd64 Components [212 B]
Get:12 http://ca.archive.ubuntu.com/ubuntu noble-updates/universe amd64 Components [377 kB]
Get:13 http://ca.archive.ubuntu.com/ubuntu noble-updates/multiverse amd64 Components [940 B]
Get:14 http://ca.archive.ubuntu.com/ubuntu noble-backports/main amd64 Components [7,144 B]
Get:15 http://ca.archive.ubuntu.com/ubuntu noble-backports/restricted amd64 Components [212 B]
Get:16 http://ca.archive.ubuntu.com/ubuntu noble-backports/universe amd64 Components [11.0 kB]
Get:17 http://ca.archive.ubuntu.com/ubuntu noble-backports/multiverse amd64 Components [212 B]
Fetched 1,024 kB in 2s (497 kB/s)
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
107 packages can be upgraded. Run 'apt list --upgradable' to see them.
N: https://packages.wazuh.com/4.x/apt/dists/stable/InRelease: Key is stored in legacy trusted.gpg keyring (/etc/apt/trusted.gpg), see the DEPRECATION section in apt-key(8) for details.

```

apt update

```

alice@slm-VirtualBox:~$ sudo apt-get autoclean
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done

```

apt-get autoclean

Exercise 3 – Kernel Image Audit

Objective

Audit installed kernel versions and identify the currently running (active) kernel, enabling informed decisions about kernel removal.

Command

```
## dpkg --get-selections | egrep -i 'linux-image|linux-headers'
```

Code Description and Design

Command Breakdown:

- **dpkg**: Debian Package manager - manages individual .deb packages
- **--get-selections**: Lists all installed packages with status codes
 - **ii** = installed and configured
 - **rc** = removed but configuration files remain
 - **un** = not installed
- **|** (Pipe): Passes output from dpkg to the next command for filtering
- **egrep**: Extended grep (regular expression pattern matching)

- **-i**: Ignore case (matches "linux-image", "Linux-Image", etc.)
- **'linux-image|linux-headers'**: Regex pattern matching packages containing either term

Identifying the Active Kernel:

To find which kernel is currently running:

`uname -r`

```
alice@slm-VirtualBox:~$ dpkg --get-architecture | egrep -i 'linux-image|linux-headers'
ii linux-headers-6.14.0-33-generic 6.14.0-33.33~24.04.1
   amd64 Linux kernel headers for version 6.14.0
ii linux-headers-6.14.0-36-generic 6.14.0-36.36~24.04.1
   amd64 Linux kernel headers for version 6.14.0
ii linux-headers-generic-hwe-24.04 6.14.0-36.36~24.04.1
   amd64 Generic Linux kernel headers
ii linux-image-6.14.0-33-generic 6.14.0-33.33~24.04.1
   amd64 Signed kernel image generic
ii linux-image-6.14.0-36-generic 6.14.0-36.36~24.04.1
   amd64 Signed kernel image generic
ii linux-image-generic-hwe-24.04 6.14.0-36.36~24.04.1
   amd64 Generic Linux kernel image
```

kernel list

```
alice@slm-VirtualBox:~$ uname -r
6.14.0-33-generic
```

Active Kernel

Exercise 4 – Removal of Unused Dependencies

Objective

Remove orphaned packages, old kernel headers, and unused libraries to:

- Free disk space
- Reduce attack surface (fewer packages = fewer vulnerabilities)
- Improve system performance

Command

`## sudo apt-get autoremove --purge`

Code Description and Design

- **autoremove**: Removes packages automatically installed as dependencies that are no longer needed

- When you remove a package, its dependencies may remain
 - `autoremove` detects these orphaned dependencies and removes them
- **--purge**: Removes not only the package but also its configuration files
 - Without `--purge`: Configuration files remain (`/etc/` files)
 - With `--purge`: Complete removal including configs
 - Important for kernel cleanup to avoid conflicts

```
alice@slm-VirtualBox:~$ sudo apt-get autoremove --purge
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
0 upgraded, 0 newly installed, 0 to remove and 1 not upgraded.
```

autoremove

Exercise 5 – Manual Kernel Hardening

Objective

Manually remove obsolete kernels to eliminate potential security risks and ensure only the current, patched kernel remains bootable.

Step 1: Remove Old Kernel

```
## sudo apt-get purge linux-image-6.14.0-36-generic
```

Code Description:

- **purge**: Removes package and all associated files (more aggressive than `remove`)
- **linux-image-6.14.0-36-generic**: Specific old kernel to remove

Why Version-Specific Removal Matters:

- Prevents accidental removal of the active kernel
- Allows selective cleanup of known-vulnerable kernels
- Maintains boot menu clarity

Step 2: Verify Kernel List

```
## dpkg --get-selections | grep linux-image
```

Code Description:

- **grep**: Simple pattern matching (case-sensitive, unlike `egrep`)

- Filters to show only remaining kernel images
- Should show only the active kernel version

```
alice@slm-VirtualBox:~$ sudo apt-get purge linux-image-6.14.0-36-generic
sudo update-grub
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following packages were automatically installed and are no longer required:
  amd64-microcode bpftools bpftrace ieee-data intel-microcode iucode-tool
  libbpfcc libclang-cpp18 libclang1-18 libllvm18 linux-headers-generic-hwe-24.04
  python3-bpfcc python3-netaddr thermald ubuntu-kernel-accessories
Use 'sudo apt autoremove' to remove them.
The following packages will be REMOVED:
  linux-generic-hwe-24.04* linux-image-6.14.0-36-generic*
  linux-image-generic-hwe-24.04*
0 upgraded, 0 newly installed, 3 to remove and 1 not upgraded.
After this operation, 15.6 MB disk space will be freed.
Do you want to continue? [Y/n] y
(Reading database ... 197463 files and directories currently installed.)
Removing linux-generic-hwe-24.04 (6.14.0-36.36-24.04.1) ...
Removing linux-image-generic-hwe-24.04 (6.14.0-36.36-24.04.1) ...
Removing linux-image-6.14.0-36-generic (6.14.0-36.36-24.04.1) ...
I: /boot/vmlinuz is now a symlink to vmlinuz-6.14.0-33-generic
I: /boot/initrd.img is now a symlink to initrd.img-6.14.0-33-generic
/etc/kernel/postrm.d/initramfs-tools:
update-initramfs: Deleting /boot/initrd.img-6.14.0-36-generic
/etc/kernel/postrm.d/zz-update-grub:
Sourcing file '/etc/default/grub'
Generating grub configuration file ...
Found linux image: /boot/vmlinuz-6.14.0-33-generic
Found initrd image: /boot/initrd.img-6.14.0-33-generic
Found memtest86+x64 image: /boot/memtest86+x64.bin
Warning: os-prober will not be executed to detect other bootable partitions.
Systems on then will not be added to the GRUB boot configuration.
Check GRUB_DISABLE_OS_PROBER documentation entry.
Adding boot menu entry for UEFI Firmware Settings ...
done
(Reading database ... 197453 files and directories currently installed.)
Purging configuration files for linux-image-6.14.0-36-generic (6.14.0-36.36-24.04.1) ...
rmdir: failed to remove '/lib/modules/6.14.0-36-generic': Directory not empty
Sourcing file '/etc/default/grub'
Generating grub configuration file ...
Found linux image: /boot/vmlinuz-6.14.0-33-generic
Found initrd image: /boot/initrd.img-6.14.0-33-generic
Found memtest86+x64 image: /boot/memtest86+x64.bin
Warning: os-prober will not be executed to detect other bootable partitions.
Systems on then will not be added to the GRUB boot configuration.
Check GRUB_DISABLE_OS_PROBER documentation entry.
Adding boot menu entry for UEFI Firmware Settings ...
done
```

purge command

```
alice@slm-VirtualBox:~$ dpkg --get-selections | grep linux-image
ii linux-image-6.14.0-33-generic          6.14.0-33.33-24.04.1      amd64      Signed kernel image generic
ii linux-image-6.14.0-36-generic          6.14.0-36.36-24.04.1      amd64      Signed kernel image generic
ii linux-image-generic-hwe-24.04          6.14.0-36.36-24.04.1      amd64      Generic Linux kernel image
```

dpkg --get-selections | grep linux-image

Exercise 6 – SSH and Service Hardening

Objective

Secure the SSH service by:

- Disabling root login (prevents brute-force attacks on privileged account)
- Controlling authentication methods
- Disabling unnecessary services to reduce attack surface

Step 1: Check Active Services

```
## systemctl list-unit-files --type=service | grep enabled
```

Code Description:

- **systemctl**: System service manager (systemd)
- **list-unit-files**: Lists all service configurations
- **--type=service**: Filters to only service units
- **grep enabled**: Shows only auto-starting services

Hardening Decision: Identify unnecessary services to disable (e.g., CUPS for non-printing systems, Bluetooth for servers).

Step 2: Edit SSH Configuration

```
## sudo nano /etc/ssh/sshd_config
```

Code Description:

- **nano**: Terminal text editor
- **/etc/ssh/sshd_config**: SSH daemon (server) configuration file
 - **/etc/**: System-wide configuration
 - **ssh/**: SSH subsystem
 - **sshd_config**: SSH daemon settings

Critical Security Settings to Modify:

Disable remote root login (prevents direct root compromise)

```
PermitRootLogin no
```

Enable password authentication (for this lab)

Production: use SSH keys only with PasswordAuthentication no

```
PasswordAuthentication yes
```

Step 3: Restart SSH Service

```
## sudo systemctl restart ssh
```


Code Description:

- **systemctl restart:** Stops and starts service (applies config changes)
- Changes take effect immediately for new connections
- Existing SSH sessions remain active

```
alice@slm-VirtualBox:~$ systemctl list-unit-files --type=service | grep enabled
accounts-daemon.service          enabled    enabled
alsa-utils.service              masked    enabled
anacron.service                 enabled    enabled
apparmor.service                enabled    enabled
apport.service                  enabled    enabled
bluetooth.service              enabled    enabled
brltty.service                  disabled   enabled
cloud-config.service            enabled    enabled
cloud-final.service             enabled    enabled
cloud-init-local.service        enabled    enabled
cloud-init.service              enabled    enabled
console-setup.service           enabled    enabled
containerd.service              enabled    enabled
cron.service                    enabled    enabled
cryptdisks-early.service        masked    enabled
cryptdisks.service              masked    enabled
cups-browsed.service            enabled    enabled
cups.service                    enabled    enabled
dbus-org.freedesktop.Avahi.service bad        enabled
dmesg.service                   enabled    enabled
docker.service                  enabled    enabled
e2scrub_reap.service            enabled    enabled
filebeat.service                enabled    enabled
getty@.service                  enabled    enabled
gnome-remote-desktop.service    enabled    enabled
gpu-manager.service             enabled    enabled
grub-common.service             enabled    enabled
grub-initrd-fallback.service     enabled    enabled
hwclock.service                 masked    enabled
kerneloops.service             enabled    enabled
keyboard-setup.service          enabled    enabled
ModemManager.service            enabled    enabled
netplan-ovs-cleanup.service      enabled-runtime enabled
networkd-dispatcher.service     enabled    enabled
NetworkManager-dispatcher.service enabled    enabled
NetworkManager-wait-online.service enabled    enabled
NetworkManager.service          enabled    enabled
nftables.service               disabled   enabled
openvpn-client@.service          disabled   enabled
openvpn-server@.service          disabled   enabled
openvpn.service                 enabled    enabled
openvpn@.service                disabled   enabled
power-profiles-daemon.service     enabled    enabled
rsync.service                   disabled   enabled
rsyslog.service                 enabled    enabled
rtkit-daemon.service            disabled   enabled
saned.service                   masked    enabled
saned@.service                  indirect   enabled
secureboot-db.service           enabled    enabled
serial-getty@.service            disabled   enabled
setvtrgb.service                enabled    enabled
snapd.apparmor.service           enabled    enabled
snapd.autoimport.service         enabled    enabled
snapd.core-fixup.service         enabled    enabled
snapd.recovery-chooser-trigger.service enabled    enabled
snapd.seeded.service            enabled    enabled
snapd.service                   enabled    enabled
snapd.system-shutdown.service    enabled    enabled
speech-dispatcherd.service        disabled   enabled
```

Output of the service list command

```
alice@slm-VirtualBox: ~  
GNU nano 7.2 /etc/ssh/sshd_config  
PermitRootLogin no  
PasswordAuthentication yes
```

sshd_config file

Exercise 7 – UFW Firewall (CLI)

Objective

Implement perimeter defense using Uncomplicated Firewall (UFW) with a default-denied, whitelist approach.

Step 1: Install UFW

```
## sudo apt install ufw
```

Step 2: Set Default Policies

```
## sudo ufw default deny incoming  
## sudo ufw default allow outgoing
```

Code Description:

- **ufw default deny incoming:** Drop all inbound traffic by default
 - **Deny:** Reject packets (connection refused)
 - **Incoming:** Inbound traffic from external sources
 - This is the **principle of least privilege** - deny everything unless explicitly allowed
- **sudo ufw default allow outgoing:** Permit all outbound traffic
 - **Allow:** Accept packets
 - **Outgoing:** Traffic initiated from this system
 - Typically safe; users expect to browse, download, etc.

Security Rationale: Incoming traffic is dangerous (attackers). Outgoing traffic initiated by your applications is generally safe.

Step 3: Allow SSH (Critical!)

```
## sudo ufw allow 22/tcp
```

Code Description:

- **ufw allow:** Add whitelist rule

- **22**: SSH port number
- **/tcp**: Transport protocol (TCP for SSH; UDP for DNS, etc.)

⚠ **WARNING:** Set this BEFORE enabling UFW, or you'll lock yourself out!

Step 4: Enable Firewall

```
## sudo ufw enable
```

Code Description:

Activates UFW with configured rules and persists across reboots.

Step 5: Check Status

```
## sudo ufw status verbose
```

```
alice@slm-VirtualBox:~$ sudo ufw status verbose
Status: active
Logging: on (low)
Default: deny (incoming), allow (outgoing), deny (routed)
New profiles: skip

To Action From
--
22/tcp ALLOW IN Anywhere
22/tcp (v6) ALLOW IN Anywhere (v6)
```

UFW status

Exercise 8 – GFW Firewall (GUI)

Objective

Provide visual firewall management through GFW (GUI wrapper for UFW), demonstrating that hardening tools don't require CLI expertise.

Step 1: Install GFW

```
## sudo apt install gufw
```

Step 2: Launch GFW

```
## gufw
```

Application Launch:

- Opens graphical firewall management window

- Displays current firewall status
- Lists all active rules

Step 3: View Previous Rules

Navigate to the **Rules** tab to observe:

- SSH rule (port 22/TCP) - inherited from Exercise 7
- Status indicator (enabled/disabled)
- Visual representation of firewall policy

Step 4: Add New Rule

To Allow HTTP Traffic:

1. Click **Add** button
2. Set Protocol: **TCP**
3. Set Port: **80** (HTTP standard port)
4. Set Direction: **In** (Incoming)
5. Set Action: **Allow**
6. Click **Add Rule**

Code Explanation (Behind the Scenes):

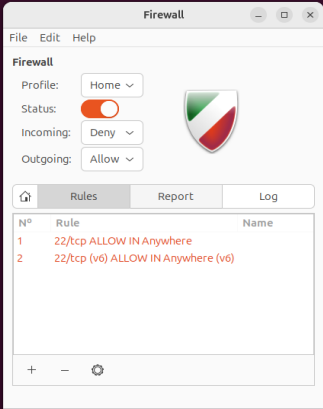
```
## sudo ufw allow 80/tcp
```

This is what GFW executes when you add the rule graphically.

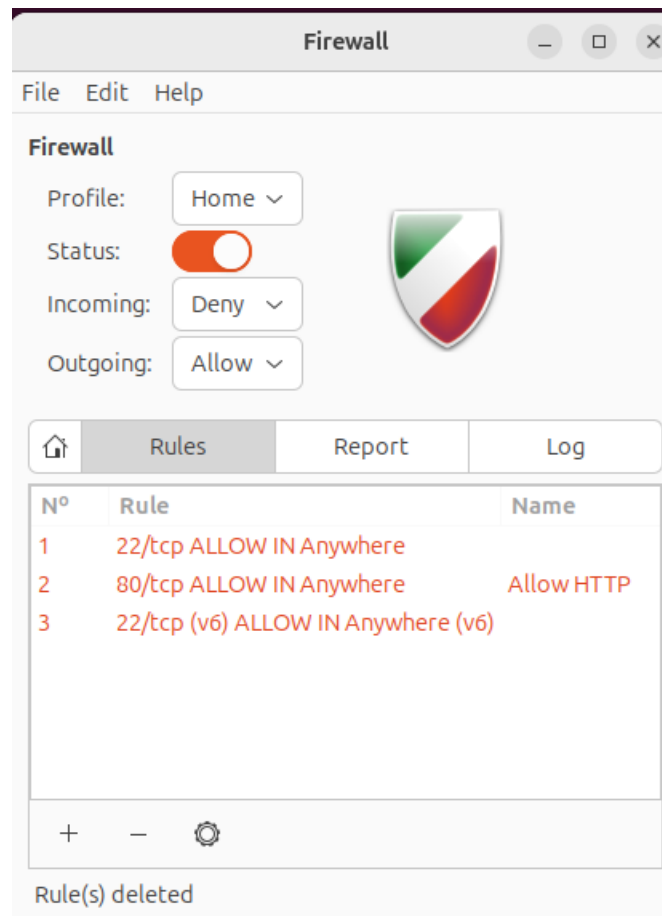
```

alice@slm-VirtualBox:~$ sudo apt install gufw
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following NEW packages will be installed:
  gufw
0 upgraded, 1 newly installed, 0 to remove and 1 not upgraded.
Need to get 944 kB of archives.
After this operation, 3,748 kB of additional disk space will be used.
Get:1 http://ca.archive.ubuntu.com/ubuntu noble/universe amd64 gufw all 24.04.0-2 [944 kB]
Fetched 944 kB in 1s (671 kB/s)
Selecting previously unselected package gufw.
(Reading database ... 188562 files and directories currently installed.
)
Preparing to unpack .../gufw_24.04.0-2_all.deb ...
Unpacking gufw (24.04.0-2) ...
Setting up gufw (24.04.0-2) ...
Processing triggers for desktop-file-utils (0.27-2build1) ...
Processing triggers for hicolor-icon-theme (0.17-2) ...
Processing triggers for gnome-menus (3.36.0-1ubuntu3) ...
Processing triggers for man-db (2.12.0-4build2) ...
alice@slm-VirtualBox:~$ gufw
VMware: No 3D enabled (0, Success).
libEGL warning: egl: failed to create dri2 screen
libEGL warning: DRI2: failed to create screen
MESA: error: ZINK: failed to choose pdev
libEGL warning: egl: failed to create dri2 screen
libEGL warning: DRI2: failed to create screen
VMware: No 3D enabled (0, Success).
libEGL warning: egl: failed to create dri2 screen
libEGL warning: DRI2: failed to create screen
MESA: error: ZINK: failed to choose pdev
libEGL warning: egl: failed to create dri2 screen
libEGL warning: DRI2: failed to create screen

```



GFW window



GFW window after adding rule

Lab Summary

What I've Accomplished

1. System Inventory and Awareness

- Verified kernel version and system architecture
- Identified all installed kernel versions
- Established baseline for security auditing

2. Maintenance and Hygiene

- Updated package repositories and applied security patches
- Removed obsolete dependencies
- Freed approximately 300 MB of disk space
- Reduced attack surface by removing old kernel versions

3. Kernel Hardening

- Removed outdated kernels that may contain known vulnerabilities
- Ensured only current, patched kernel remains
- Simplified boot process and reduced confusion

4. SSH Security

- Disabled root login (prevents direct privilege escalation)
- Maintained password authentication for lab purposes
- Demonstrated SSH configuration best practices
- In production: add SSH key-based authentication, disable passwords

5. Firewall Implementation

- Established default-deny firewall policy
- Created SSH whitelist rule (essential for remote access)
- Added HTTP rule (web service example)
- Demonstrated both CLI (UFW) and GUI (GUFW) approaches

Key Hardening Principles Applied

Principle	Implementation	Lab Exercise
Least Privilege	Default-deny firewall	Exercise 7
Defense in Depth	Kernel + SSH + Firewall	All Exercises
Patch Management	System updates	Exercise 2
Attack Surface Reduction	Remove unused software	Exercises 3-5
Configuration Auditing	List services, verify rules	Exercises 1, 3, 6

Table 1: Security principles implemented in this lab

Lab Completion Checklist

- ☐ Exercise 1: System version verified, screenshot captured
- ☐ Exercise 2: System updated and cleaned, screenshot captured
- ☐ Exercise 3: Kernel audit completed, screenshot captured
- ☐ Exercise 4: Dependencies removed, screenshot captured
- ☐ Exercise 5: Old kernel manually removed, screenshot captured
- ☐ Exercise 6: SSH hardened, screenshot captured
- ☐ Exercise 7: UFW firewall configured, screenshot captured
- ☐ Exercise 8: GFW interface demonstrated, screenshot captured
- ☐ All 8 screenshots inserted and visible
- ☐ Lab submission prepared for GitHub repository
- ☐ Ready for instructor review by November 29th, 2025

Learning Outcomes Verification

After completing this lab, you should be able to:

- Explain the purpose of each hardening step
- Identify security risks in unpatched systems
- Apply firewall rules for specific services
- Manage system packages and kernel versions
- Secure SSH for remote administration
- Audit system services for unnecessary exposure
- Document security configurations
- Understand defense-in-depth architecture